

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа № 5
Межсервисное взаимодействие через RabbitMQ

Выполнил:

Колесников Игорь Евгеньевич

БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Для событий жизненного цикла отклика используется RabbitMQ. application-service публикует события, а notification-service получает их асинхронно. Брокер и оба сервиса определены в labs/lab23/docker-compose.yml.

Вариант проекта: Jobby - сервис поиска работы.

Ход работы

Топология

Параметр	Значение
Exchange	jobby, тип topic, durable
Publisher	application-service
Consumer	notification-service
Routing key создания отклика	application.created
Очередь создания отклика	q.application.created, durable
Routing key изменения статуса	application.status_changed
Очередь изменения статуса	q.application.status, durable

В опубликованных сообщениях включён флаг persistent. Consumer устанавливает prefetch(1) и подтверждает обработанное сообщение через ack. Сейчас результат обработки выводится в лог notification-service; отправки email или push-уведомлений нет.

Формат сообщений

При создании отклика отправляется событие:

```
JSON • пример сообщения

1  {
2    "event": "application.created",
3    "applicationId": "uuid",
4    "vacancyId": "uuid",
5    "resumeId": "uuid",
6    "seekerUserId": "uuid",
7    "employerUserId": "uuid",
8    "createdAt": "2026-09-26T00:00:00.000Z"
9  }
```

Рисунок 1 - Пример JSON-сообщения

При изменении статуса отклика отправляется событие:

```
JSON • пример сообщения

1  {
2    "event": "application.status_changed",
3    "applicationId": "uuid",
4    "newStatus": "INVITED",
5    "seekerUserId": "uuid",
6    "updatedAt": "2026-09-26T00:00:00.000Z"
7  }
```

Рисунок 2 - Пример JSON-сообщения

Проверка

1. Запустить микросервисы командой `docker compose up --build -d` из `labs/lab23` при свободных портах 3101-3105, 5672 и 15672.
2. Создать работодателя, вакансию, соискателя и резюме через API.
3. Создать отклик запросом `POST /api/v1/applications` в `application-service` на порту 3104.
4. Просмотреть `docker compose logs notification-service` и найти сообщение `application.created`.
5. Изменить статус запросом `PATCH /api/v1/applications/{applicationId}/status` и найти в журнале `application.status_changed`.

RabbitMQ Management UI доступен на <http://localhost:15672> с учётными данными из `docker-compose.yml`. В разделе **Queues and Streams** видны обе очереди и количество сообщений.

Ограничение реализации

Сохранение отклика в PostgreSQL и публикация сообщения выполняются отдельными операциями. Для гарантированной доставки при сбое между ними понадобился бы `transactional outbox`; в текущей учебной реализации его нет.

Вывод

RabbitMQ подключён, события создания отклика и изменения его статуса передаются от `application-service` к `notification-service`. Их обработка подтверждается сообщениями в журнале `consumer`.